



Computing the longest common almost-increasing subsequence

Mohammad Tawhidul Hasan Bhuiyan, Muhammad Rashed Alam, M. Sohel Rahman*

Department of CSE, BUET, ECE Building, West Palashi, Dhaka-1205, Bangladesh

ARTICLE INFO

Article history:

Received 27 August 2021
Received in revised form 9 July 2022
Accepted 19 July 2022
Available online 3 August 2022
Communicated by T. Calamoneri

Keywords:

Algorithms
Longest common subsequence
Longest increasing subsequence

ABSTRACT

In this paper, we revisit the problem of computing **longest common almost increasing subsequence (LCAIS)** where, given two input sequences, the goal is to compute a common subsequence that is ‘almost’ increasing. Here the concept of an almost increasing subsequence offers an interesting relaxation over the increasing condition. This problem has been studied in the literature **albeit** with some constraints. Here, we present a number of a number of algorithmic approaches to solve the problem more generally and efficiently.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Given a sequence as input, the longest increasing subsequence (LIS) problem seeks to find a subsequence having the highest possible length, in which the subsequence elements are in sorted order, lowest to highest. This classic problem was first tackled by Robinson [3]. Schensted [25] gave the classical dynamic-programming algorithm for the problem, which appears in many algorithmic textbooks. This algorithm runs in $O(n \log k)$, where n is the length of the input sequence, and k is the length of the longest increasing subsequence (i.e., output size). Knuth [19] gave generalizations to the problem with relations to Young tableaux. For the comparison-tree model, Fredman [12] showed that $O(n \log n)$ comparisons are both necessary and sufficient, to find the length or produce the subsequence. Here it assumes the worst case number of comparisons (hence, $k \sim n$, making it consistent with [25]). On integer alphabets, the fastest known solution runs in $O(n \log \log n)$ time (see [28] and references therein); it relies on the priority search trees of van Emde Boas [11] that can provide **$O(\log \log n)$ amortized time** per operation when keys are drawn from the set $\{1, 2, \dots, n\}$. Liben-Nowell et al. [21] have explored the LIS problem in the streaming model which typically aims at reducing the memory space required by the computation to poly-logarithmic amount, in addition to achieving an efficient running time (see also [24]).

A natural extension of the LIS problem is the Longest Common Increasing Subsequence (LCIS) problem [28] that follows from the classic Longest Common Subsequence (LCS) problem. In the LCS problem, given two (or more) input sequences, we are interested to compute a common subsequence of the highest length. LCS [7,15,14] and variants thereof (e.g., [27,6,15,13,16,18,17]) have received significant attention in the literature due to their diverse applications in different branches of science and engineering. **It has been proven that there is no truly subquadratic algorithm for this problem unless the Strong Exponential Time Hypothesis (SETH) fails** [1,5]. This also holds for approximate solutions to LCS [2]. In fact, it should be

* Corresponding author.

E-mail address: msrahman@cse.buet.ac.bd (M.S. Rahman).

noted here that the lower bound for approximating LCS is conditioned on some hypothesis much stronger SETH. Motivated by this result, Bringman et al. studied the multivariate complexity of LCS [4]. They proved the optimal running time for LCS under SETH as $(n + \min\{d, \delta\Delta, \delta m\})^{1 \pm o(1)}$, where n and m are the length of the longer and shorter sequence respectively; d is the number of dominant pairs¹; and with ℓ being the length of the LCS, $\delta = n - \ell$, $\Delta = m - \ell$ are the number of deletions in two sequences. For LCIS as well we seek to find the common subsequence but with the added condition that the computed common subsequence must be an increasing subsequence. Similar to the LCS problem, it has been proven that there is no truly subquadratic algorithm for this problem unless SETH fails [8]. Hence, works have been done on algorithms with output-dependant running time. Kutz et al. [20] provided an algorithm with output-dependant running time of $O((n + m\ell) \log \log \sigma + \text{Sort})$, where n and m are the length of the longer and shorter sequence respectively; ℓ is the length of the LCIS; σ is the size of alphabet; and Sort is the time to sort each sequence.

Yet another variant of LIS, albeit from a different angle, is the **Longest Almost-Increasing Subsequence (LAIS) problem**. Introduced by Elmasry in [9], this is a relaxed version of the LIS problem: for a given sequence $X = \langle x_1, x_2, \dots, x_n \rangle$ and a constant $c \geq 0$, the goal here is to construct a longest subsequence $S = \langle s_1 s_2 \dots s_\ell \rangle$ of X such that $\forall i \in [2.. \ell] : \max_{j=1}^{i-1} s_j < s_i + c$. Now, S here can be thought of as an *almost* increasing subsequence of X . Elmasry gave an algorithm that runs in **$O(n \log \ell)$ time** which is asymptotically optimal for the comparison-tree model (as this algorithm can find LIS by setting $c = 0$). The algorithm uses pointer-based data structures, namely, jump lists and AVL trees.

The LCAIS problem, first introduced and studied in [23], is a natural extension of LAIS and LCIS problem. It takes two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ and a constant $c > 0$ as input and computes a longest common subsequence $S = \langle s_1, s_2, \dots, s_\ell \rangle$ of X and Y such that $\forall i \in [2.. \ell] : \max_{j=1}^{i-1} s_j < s_i + c$. In fact, in [23], a slightly restricted variant of LCAIS was studied: **it was assumed that the given sequences, X and Y , do not have any repeated elements and each element is an integer**. They claimed to present an algorithm that runs in **$O(n(m + c^2))$ time**. Note that, LCAIS with the case, $c = 0$ actually reduces to the problem of LCIS. This is why here we have $c > 0$ rather than $c \geq 0$.

During the publication and review process of the current paper we came to know of a very recent work of Ta et al. [26], where the authors presented a dynamic programming algorithm that can compute **an LCAIS between any two sequences with repeated elements in $O(nm\ell)$ time and $O(nm)$ space**, where n and m are the lengths of two input sequences and ℓ is the length of the output LCAIS. In fact, as has been reported in [26], Ta et al. first identified a flaw in the algorithm of [23], which motivated them to study this problem and that too in a more general setting than that of [23]. Thus the work of Ta et al. [26] can be seen as the current state of the art for the LCAIS algorithms.

In this paper, we revisit the LCAIS problem and present a number of algorithmic approaches to solve the problem efficiently. In particular, we discuss **three different approaches**, offering different trade-offs, to solve the problem (Sections 3–5). The first approach (Section 3) provides a straightforward way to solve the problem which acts as a baseline algorithm for the subsequent approaches. The second approach (Section 4) improves upon the baseline algorithm and works best when there are fewer elements within the range, c (which might be true in some practical cases). We present four different implementations of this approach with different degrees of improvements: first we give a naive implementation, the second and third implementations improve upon that and then the fourth implementation tries a different approach to improve upon the naive implementation. The third approach (Section 5) again tries to improve upon the baseline algorithm and works best when the number of matches between two sequences is low; **it achieves $O(mn)$ time complexity when elements are repeated at most a constant number of times**. Subsequently, we further discuss how to restore an LCAIS in $O(mn)$ space (Section 6). Finally, we compare our algorithms with the state of the art, i.e., the work of [26] and discuss some interesting future research avenues (Section 7).

2. Definitions and terminologies

Definition 1 (Prefix). For a given sequence $X = \langle x_1, x_2, \dots, x_m \rangle$, the i^{th} prefix of X is denoted by $X_i = \langle x_1, x_2, \dots, x_i \rangle$ where $i = 0, 1, \dots, m$.

For example, if $X = \langle 1, 5, 1, 3, 2, 4 \rangle$, then $X_3 = \langle 1, 5, 1 \rangle$ and X_0 is the empty sequence. Please note that we will be using capital letters to denote sequences and small letters to denote a symbol within a sequence. Therefore, X and X_i would mean a sequence and the i^{th} prefix thereof whereas x_i would mean the i^{th} symbol of X .

Note that, all sequences should be assumed to consist of real numbers.

Definition 2 (MSeq). For two given sequences X and Y , we define a sequence $Z = \langle z_1, z_2, \dots, z_p \rangle$ as the **intersection** of the set of elements in X and Y **in increasing order**. We refer to Z as the **max sequence** (MSeq) of X and Y . For convenience, we define $z_0 = -\infty$. Hence, we have $z_0 < z_1 < z_2 < \dots < z_p$.

存在A和B的數字且排序過

We can generate Z by sorting X and Y , comparing all pairs, and keeping the matched elements with no repetition. This runs in $O(mn)$ time. It is possible to get a better algorithm that runs in **$O(m + n + \text{Sort})$ time**, where Sort is the required time to sort X and Y . However, this does not improve out overall running time asymptotically.

¹ A pair (i, j) dominates a pair (i', j') if we have $i \leq i'$ and $j \leq j'$.

Definition 3 (Index-Map). If Z is the MSeq of X and Y , then for a real number b , we define its index-map with respect to Z , $\mu_Z(b)$ as follows:

$$\mu_Z(b) = \begin{cases} 0 & : b \notin Z \\ t & : b \in Z \text{ and } b = z_t \quad \text{b在Z的index} \end{cases} \quad (1)$$

We can calculate $\mu_Z(b)$ through a single iteration over Z . Therefore, to generate index-map for all elements of Y , it takes $O(np)$ time. But it is possible to get a better algorithm that runs in $O(n + \text{Sort})$ time where Sort is the required time to sort Y . However, this does not improve out overall running time asymptotically. Notably, for the sake of ease, we may use a simpler notation for the Index-Map in subsequent sections as follows: we will use μ_j to denote $\mu_Z(y_j)$ (Sections 3 and 4), and μ_i to denote $\mu_Z(x_i)$ (Section 5).

Definition 4 (Upper-Index-Map). If Z is the MSeq of X and Y , and $c > 0$ is a given constant, then for a real number b , we define its upper-index-map with respect to Z , c , namely, $\eta_{Z,c}(b)$ as follows:

$$\eta_{Z,c}(p) = \max\{k \mid 0 \leq k \leq p, z_k < b + c\} \quad (2)$$

We can calculate $\eta_{Z,c}(b)$ through a single iteration over Z . Therefore, to generate upper-index-map for all elements of Y , it takes $O(np)$ time. But it is possible to get a better algorithm that runs in $O(n + \text{Sort})$ time where Sort is the required time to sort Y . However, this does not improve out overall running time asymptotically. Notably, a simpler notation will be used for Upper-Index-Map in the subsequent sections. We will use η_j to denote $\eta_{Z,c}(y_j)$ (Sections 3 and 4) and η_i to denote $\eta_{Z,c}(x_i)$ (Section 5). The following lemmas, which will be useful later, easily follow from the above definitions.

Lemma 1. $z_{\mu_Z(b)} \leq b$.

Lemma 2. If $b \in Z$, then $z_{\mu_Z(b)} = b$.

Lemma 3. For all $1 \leq k \leq p$, if $k \leq \eta_{Z,c}(b)$, then $z_k < b + c$.

Lemma 4. For all $1 \leq k \leq p$, if $k > \eta_{Z,c}(b)$, then $z_k \geq b + c$.

Lemma 5. If $S = \langle s_1, s_2, \dots, s_\ell \rangle$ is an almost-increasing sequence, then for all $1 \leq t \leq \ell$, $\max(S_t) \leq z_{\eta_{Z,c}(s_t)}$.

3. Approach 1: tracking the maximum

In this section we present our first approach to solve the LCAIS. If we want to append an element to the end of an existing almost-increasing sequence and keep it almost-increasing, we need to check the maximum element of that sequence. We can append the element, if and only if the maximum element is bigger than the appending element by less than c amount. So, for an element, we need to keep track of other elements that are bigger than it by less than c amount. To do it efficiently, we can keep track of only the maximum of such element, more specifically, the index thereof in Z . In this section, we leverage this idea to develop a solution. Before we proceed, recall that μ_j and η_j are shorthand for $\mu_Z(y_j)$ and $\eta_{Z,c}(y_j)$ respectively.

3.1. Recursive formulation

Theorem 1. Suppose $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ are two input sequences and $Z = \langle z_1, z_2, \dots, z_p \rangle$ is their MSeq. If we define $\theta_c[i, j, k]$ to be the length of an LCAIS of the sequences X_i and Y_j that has maximum element less than or equal to z_k , then $\theta_c[i, j, k]$ can be calculated by the following formula.

$$\theta_c[i, j, k] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \text{ or } k = 0, \\ \theta_c[i - 1, j - 1, k] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \text{ and } \mu_j \leq k \leq \eta_j, \\ \max(\theta_c[i - 1, j - 1, k], \theta_c[i, j, \eta_j]) & \text{if } i, j > 0 \text{ and } x_i = y_j \text{ and } k > \eta_j, \\ \max(\theta_c[i, j - 1, k], \theta_c[i - 1, j, k]) & \text{if } i, j > 0 \text{ and } (x_i \neq y_j \text{ or } k < \mu_j). \end{cases} \quad (3)$$

For convenience, we assume that the maximum element of an empty sequence is $-\infty$.

Proof. Assume $\theta_c[i, j, k] = \ell$, and $S = \langle s_1, s_2, \dots, s_\ell \rangle$ is a corresponding sequence. So, S is an LCAIS of X_i and Y_j such that $\max(S) \leq z_k$.

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0	0	0	0	1	1
2	2	0	0	0	0	1	1
3	3	0	0	0	1	1	1
4	1	0	1	1	2	2	2
5	2	0	1	1	2	2	2
6	5	0	1	1	2	2	3
7	3	0	1	1	2	2	3
8	1	0	1	1	2	2	3

(a) $i = 4; x_i = 5$

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0	0	0	0	1	1
2	2	0	0	1	1	2	2
3	3	0	0	1	1	2	2
4	1	0	1	1	2	2	2
5	2	0	1	2	3	3	3
6	5	0	1	2	3	3	3
7	3	0	1	2	3	3	3
8	1	0	1	2	3	3	3

(b) $i = 5; x_i = 2$

Fig. 1. A slice of the table θ_c while calculating LCAIS by Algorithm 1 with $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$. Here, **red** cell represents $z_k < y_j$, **green** cell represents $y_j \leq z_k < y_j + c$ and **blue** cell represents $z_k \geq y_j + c$. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

(1) It is trivial that $\theta_c[i, j, k] = 0$, if $i = 0$ or $j = 0$, as X_0 and Y_0 are empty sequences. And $k = 0$ implies a sequence with maximum element being less than or equal to $z_0 = -\infty$, which has to be empty. So, $\theta_c[i, j, k] = 0$ when $k = 0$.

(2) For the second case, $x_i = y_j$ and $\mu_j \leq k \leq \eta_j$. So, $z_k \geq z_{\mu_j} = y_j$ (by Lemma 2) = x_i and $z_k \leq z_{\eta_j} < y_j + c$ (by Lemma 3). Now if $s_\ell \neq x_i$, then we could append $x_i = y_j$ to S to obtain a common almost-increasing subsequence (CAIS) S' of X_i and Y_j of length- $(\ell + 1)$ with z_k as maximum element (because $z_k < y_j + c$ implies it is a valid CAIS and $z_k \geq y_j$ implies $\max(S') \leq z_k$), contradicting the supposition that S is an LCAIS of X_i and Y_j with maximum element being at most z_k . Thus, we must have $s_\ell = x_i = y_j$. Now, the prefix $S_{\ell-1}$ is a length- $(\ell - 1)$ CAIS of X_{i-1} and Y_{j-1} such that $\max(S_{\ell-1}) \leq z_k$. We wish to show that it is an LCAIS (which implies $\theta_c[i - 1, j - 1, k] = \ell - 1$). Suppose for the purpose of contradiction that there exists a CAIS W of X_{i-1} and Y_{j-1} such that $\max(W) \leq z_k$ with length greater than $\ell - 1$. Then, appending $x_i = y_j$ to W produces a CAIS of X_i and Y_j with maximum element at most z_k whose length is greater than ℓ , which is a contradiction.

Hence, $\theta_c[i, j, k] = \ell = \theta_c[i - 1, j - 1, k] + 1$.

(3) For the third case, $x_i = y_j$ and $k > \eta_j$. So, $z_k \geq y_j + c$ (by Lemma 4). Now, we can consider two sub-cases: (3.1) $s_\ell = y_j$; (3.2) $s_\ell \neq y_j$.

(3.1) If $s_\ell = y_j$, then $\max(S) \leq z_{\eta_j}$ (by Lemma 5) $< z_k$. So, S is an LCAIS of X_i and Y_j having maximum element at most z_{η_j} . Again, $\max(S) < z_k$ implies S is also an LCAIS of X_i and Y_j having maximum element at most z_k . So, $\theta_c[i, j, k] = \theta_c[i, j, \eta_j]$.

(3.2) If $s_\ell \neq y_j$, then S is an LCAIS of X_{i-1} and Y_{j-1} such that $\max(S) \leq z_k$. So, $\theta_c[i, j, k] = \theta_c[i - 1, j - 1, k]$.

Considering the two sub-cases we get $\theta_c[i, j, k] = \max(\theta_c[i - 1, j - 1, k], \theta_c[i, j, \eta_j])$.

(4) If $x_i \neq y_j$, then we can consider two sub-cases: (4.1) $s_\ell \neq x_i$ and (4.2) $s_\ell = x_i$. And if $x_i = y_j$ and $k < \mu_j$, then $s_\ell \leq z_k < z_{\mu_j} = y_j$ (by Lemma 2) = x_i , i.e., $s_\ell \neq x_i$ which is the same as (4.1). We handle the two sub-cases below.

(4.1) If $s_\ell \neq x_i$, then S is a CAIS of X_{i-1} and Y_j such that $\max(S) \leq z_k$. If there were a CAIS W of X_{i-1} and Y_j with length greater than ℓ such that $\max(W) \leq z_k$, then W would also be a CAIS of X_i and Y_j with $\max(W) \leq z_k$, contradicting the assumption that S is an LCAIS of X_i and Y_j with $\max(S) \leq z_k$. So, $\theta_c[i, j, k] = \theta_c[i - 1, j, k]$.

(4.2) If $s_\ell = x_i$, then $s_\ell \neq y_j$. So, by symmetry to the previous sub-case, we can prove that $\theta_c[i, j, k] = \theta_c[i, j - 1, k]$.

Considering the two sub-cases we get $\theta_c[i, j, k] = \max(\theta_c[i, j - 1, k], \theta_c[i - 1, j, k])$.

The proof is complete. $\square$

Following the definition of θ , we can see that $\theta_c[m, n, p]$ gives us the length of an LCAIS of X and Y .

Before concluding this section, a brief discussion is in order. We note that, “an LCAIS of the sequences X_i and Y_j that has the maximum element less than or equal to z_k ” should be interpreted as a CAIS S of X_i and Y_j that satisfies $\max(S) \leq z_k$ and has maximum length among such CAISes satisfying $\max(S) \leq z_k$. In particular, there could be longer CAISes of X_i and Y_j (but they do not satisfy $\max(S) \leq z_k$).

3.2. Implementation

A straightforward implementation of the DP formulation of Theorem 1 is given in Algorithm 1. We simply keep a table θ_c of dimension $(m + 1) \times (n + 1) \times (p + 1)$, such that $\theta_c[i, j, k]$ is as defined in Theorem 1. For each $i, j, k > 0$, $\theta_c[i, j, k]$ depends only on whether $x_i = y_j$ and the values, $\theta_c[i - 1, j - 1, k]$, $\theta_c[i, j, \eta_j]$ (when $k > \eta_j$), $\theta_c[i - 1, j, k]$ and $\theta_c[i, j - 1, k]$, which are all calculated before $\theta_c[i, j, k]$. This algorithm returns $\theta_c[m, n, p]$, which is the length of an LCAIS of X and Y . It is easy to see that Algorithm 1 runs in $O(mnp)$ time and space.

Fig. 1a is a slice of the table θ produced by Algorithm 1 with $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$ (So, $Z = \langle 1, 2, 3, 4, 5 \rangle$). The left (right) table in the figure is for $i = 4$ ($i = 5$).

Algorithm 1 LCAIS: tracking the maximum.

```

1: function LCAIS( $X, Y, c$ )
2:    $Z \leftarrow \text{MSeq}(X, Y)$  ▷ calculate MSeq as described in Definition 2
3:    $M \leftarrow \text{INDEXMAP}(Z, Y)$  ▷  $M = \langle \mu_1, \dots, \mu_n \rangle : \mu_j = \mu_Z(y_j)$ 
4:    $H \leftarrow \text{UPPERINDEXMAP}(Z, Y, c)$  ▷  $H = \langle \eta_1, \dots, \eta_n \rangle : \eta_j = \eta_{Z,c}(y_j)$ 
5:   let  $\theta_c[0..m, 0..n, 0..p]$  be new table
6:   for  $j \leftarrow 0$  to  $n$  do
7:     for  $k \leftarrow 0$  to  $p$  do
8:        $\theta_c[0, j, k] \leftarrow 0$ 
9:   for  $i \leftarrow 1$  to  $m$  do
10:    for  $k \leftarrow 0$  to  $p$  do
11:       $\theta_c[i, 0, k] \leftarrow 0$ 
12:    for  $j \leftarrow 1$  to  $n$  do
13:       $\theta_c[i, j, 0] \leftarrow 0$ 
14:      for  $k \leftarrow 1$  to  $p$  do
15:

$$\theta_c[i, j, k] \leftarrow \begin{cases} \theta_c[i-1, j-1, k] + 1, & \text{if } x_i = y_j \text{ and } \mu_j \leq k \leq \eta_j \\ \max(\theta_c[i-1, j-1, k], \theta_c[i, j, \eta_j]), & \text{if } x_i = y_j \text{ and } k > \eta_j \\ \max(\theta_c[i, j-1, k], \theta_c[i-1, j, k]), & \text{otherwise} \end{cases}$$

16:   return  $\theta_c[m, n, p]$ 

```

4. Approach 2: tracking the last element

In addition to keeping track of the maximum elements, as has been done in the previous section, we now present a different formulation through keeping track of the last elements of the sequences (i.e., index thereof). Here, we will treat the input sequences asymmetrically. We will consider **the subsequences that end on a specific index of one input sequence.**

4.1. Recursive formulation

Theorem 2. Suppose $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ are two input sequences and $Z = \langle z_1, z_2, \dots, z_p \rangle$ is their MSeq. If we define $f_c[i, j, k]$ to be the length of an LCAIS of the sequences X_i and Y_j that ends on y_j and has maximum element less than or equal to z_k , then $f_c[i, j, k]$ can be computed by the following dynamic programming formula.

$$f_c[i, j, k] = \begin{cases} 0, & \text{if } i = 0 \text{ or } j = 0 \text{ or } k < \mu_j, \\ f_c[i, j, \eta_j], & \text{if } i, j > 0 \text{ and } x_i = y_j \text{ and } k > \eta_j, \\ \max_{0 \leq t < j} (f_c[i-1, t, k]) + 1, & \text{if } i, j > 0 \text{ and } x_i = y_j \text{ and } \mu_j \leq k \leq \eta_j, \\ f_c[i-1, j, k], & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases} \quad (4)$$

For convenience, we assume that the maximum element of an empty sequence is $-\infty$ and it ends on y_j for all $j \in [0..n]$.

Proof. Assume $f_c[i, j, k] = \ell$, and $S = \langle s_1, s_2, \dots, s_\ell \rangle$ is a corresponding sequence. So, S is an LCAIS of X_i and Y_j such that S ends on y_j ($s_\ell = y_j$ if $\ell > 0$) and $\max(S) \leq z_k$.

(1) It is trivial that $f_c[i, j, k] = 0$ when $i = 0$ or $j = 0$, as X_0 and Y_0 are empty sequences. And $k < \mu_j$ implies $z_k < z_{\mu_j} \leq y_j$ (by Lemma 1) which means $f_c[i, j, k]$ is the length of a sequence that ends on y_j and has maximum element less than y_j which is impossible. So, $f_c[i, j, k] = 0$ when $k < \mu_j$.

In the next two cases (2) and (3), $x_i = y_j$ and $k \geq \mu_j$. Here, S has to be nonempty as $\langle y_j \rangle$ is a valid CAIS of X_i and Y_j that ends on y_j and $\max(\langle y_j \rangle) = y_j = z_{\mu_j}$ (by Lemma 2) $\leq z_k$ (as $k \geq \mu_j$). This means, in these cases $s_\ell = y_j$.

(2) For this case, $x_i = y_j$ and $k > \eta_j$. Here, $s_\ell = y_j$ which implies $\max(S) \leq z_{\eta_j}$ (by Lemma 5) $< z_k$. So, S is an LCAIS of X_i and Y_j that ends on y_j and has maximum element at most z_{η_j} . Again, $\max(S) < z_k$ implies S is also an LCAIS of X_i and Y_j that ends on y_j and has maximum element at most z_k . So, $f_c[i, j, k] = f_c[i, j, \eta_j]$.

(3) Now we consider $x_i = y_j$ and $\mu_j \leq k \leq \eta_j$. We need to show the following two equations,

$$\forall t < j : f_c[i-1, t, k] \leq \ell - 1 \quad (5)$$

$$\exists t < j : f_c[i-1, t, k] = \ell - 1 \quad (6)$$

(3.1) Suppose there exists some t' such that $f_c[i-1, t', k] > \ell - 1$. Now, we can append $x_i = y_j$ to that sequence so that $f_c[i, j, k] > \ell$, a contradiction. This proves Equation (5).

(3.2) By definition, there exists some $t < j$ such that $S_{\ell-1}$ is a CAIS that ends on y_t and $\max(S_{\ell-1}) \leq z_k$. Therefore, $f_c[i-1, t, k] \geq \ell - 1$. Using the inequality from (3.1), we can say, $f_c[i-1, t, k] = \ell - 1$. This proves Equation (6).

Finally, Equations (5), (6) imply, $f_c[i, j, k] = \max_{0 \leq t < j} (f_c[i-1, t, k]) + 1$.

(4) If $x_i \neq y_j$, then $s_\ell \neq x_i$. So, S is an LCAIS of X_{i-1} and Y_j that ends on y_j and has maximum element at most z_k . Hence, $f_c[i, j, k] = f_c[i-1, j, k]$.

The proof is complete. $\square$

The following easy corollaries will be useful later to prove some theorems and corollaries.

Corollary 1. $\forall i, j, k > 1 : f_c[i, j, k] \geq f_c[i, j, k-1]$ **k的部分是遞增的**

Corollary 2. $\forall i > 0, j, k : f_c[i, j, k] \geq f_c[i-1, j, k]$ **i的部分是遞增的**

Furthermore, it is crucial to note that f_c and θ_c are related as captured by the following theorem (Theorem 3); the proof follows readily from the respective definitions. Recall that, $f_c[i, j, k]$ has a restriction that the sequence has to end on y_j whereas $\theta_c[i, j, k]$ does not.

Theorem 3. $\theta_c[i, j, k] = \max_{0 \leq t \leq j} f_c[i, t, k]$ $\square$

It readily follows from Theorem 3 that the length of an LCAIS of X and Y can be computed as follows:

$$\theta_c[m, n, p] = \max_{0 \leq j \leq n} f_c[m, j, p] \quad (7)$$

In the rest of this section we will discuss several implementations of the current approach. Before diving into implementation details, we first define some notations for the sake of ease in presentation and analysis.

- m = length of X
- n = length of Y
- p = length of Z
[Z contains each unique element that exists in both X and Y once. So, $p \leq \min(m, n)$.]
- $c_p = \max_{1 \leq k \leq p} |\{t \mid z_k \leq z_t < z_k + c\}|$
[c_p will be a constant $\leq c$ if the elements in Z are integers]
- $c_n = \max_{1 \leq j \leq n} |\{t \mid y_j \leq y_t < y_j + c\}|$
[$c_n \leq c$ when the elements in Y are integers]
- $\mathcal{M}$ = number of matches between X and Y , i.e., number of pairs (i, j) such that $x_i = y_j$
- ℓ = length of calculated LCAIS

4.2. A simple implementation

For implementation purposes, it will be useful to rewrite Theorem 3 recursively as follows.

Corollary 3.

$$\theta_c[i, j, k] = \begin{cases} 0, & \text{if } j = 0 \\ \max(\theta_c[i, j-1, k], f_c[i, j, k]), & \text{if } j > 0 \end{cases} \quad (8)$$

We present a simple implementation of it in Algorithm 2 where we calculate $f_c[i, j, k]$ and $\theta_c[i, j, k]$ for all combinations of i, j and k . Here, $F[j, k]$ represents $f_c[i, j, k]$ and $T[k]$ represents $\theta_c[i, j, k]$ at the end of iteration i, j . Initially (before the loop starts) i is taken to be 0 and similar for j . Hence, both F and T are initialized with zeros (from Equation (4)). Now at the start of iteration j , $T[k] = \theta_c[i-1, j-1, k]$. In Line 9, T is copied into T' and T' does not change after that. So, $T'[k] = \theta_c[i-1, j-1, k]$. Line 11 follows from Equation (8). Lines 12–16 follow from Equation (4). Finally, Line 17 follows from Equation (7).

Clearly, the running time of this algorithm is $O(mnp)$ as in total mnp entries are calculated. The space complexity is $O(np)$ which is the size of the table F .

Fig. 2a shows an illustration of the current simple implementation of LCAIS considering $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$. Here, T (Table b) is calculated using F (Table a) and then that T is used to calculate new F (Table c). At this point a brief discussion is in order based on the observations on table representing F : red cells always contain 0 and blue cells on a row are equal to the rightmost green cell on that row. These observations come directly from Theorem 2: first one from case 1 and second one from case 2. We will use these observations to improve upon our implementation in the next section.

Algorithm 2 LCAIS: tracking the last element - simple implementation.

```

1: function LCAIS( $X, Y, c$ )
2:    $Z \leftarrow \text{MSEQ}(X, Y)$ 
3:    $M \leftarrow \text{INDEXMAP}(Z, Y), H \leftarrow \text{UPPERINDEXMAP}(Z, Y, c)$ 
4:   Initialize a matrix  $F$  of dimension  $(n+1) \times p$  with 0
5:   for  $i \leftarrow 1$  to  $m$  do
6:     Initialize an array  $T$  of size  $p$  with 0
7:     for  $j \leftarrow 1$  to  $n$  do
8:       if  $x_i = y_j$  then
9:          $T' \leftarrow T$ 
10:      for  $k \leftarrow 1$  to  $p$  do
11:         $T[k] \leftarrow \max(T[k], F[j, k])$ 
12:      if  $x_i = y_j$  then
13:        for  $k \leftarrow \mu_j$  to  $\eta_j$  do
14:           $F[j, k] \leftarrow T'[k] + 1$ 
15:        for  $k \leftarrow \eta_j + 1$  to  $p$  do
16:           $F[j, k] \leftarrow F[j, \eta_j]$ 
17:   return  $\max_{0 \leq j \leq n} F[j, p]$ 

```

4.3. An improved implementation: reducing the search space

Based on the observations made at the end of the previous section, we only need to use the value of $f_c[i, j, k]$ for $\mu_j \leq k \leq \eta_j$. Hence, the search space reduces down to the following: $i \in [0..m]$, $j \in [0..n]$, $k \in [\mu_j.. \eta_j]$. In this case, we need θ_c values to be calculated using $f_c[i, j, k]$ where $\mu_j \leq k \leq \eta_j$. Therefore, from Equation (4) and Corollary 3, we have the following corollary.

Corollary 4.

$$\theta_c[i, j, k] = \begin{cases} 0, & \text{if } j = 0 \\ \theta_c[i, j-1, k], & \text{if } j > 0 \text{ and } k < \mu_j \\ \max(\theta_c[i, j-1, k], f_c[i, j, k]), & \text{if } j > 0 \text{ and } \mu_j \leq k \leq \eta_j \\ \max(\theta_c[i, j-1, k], f_c[i, j, \eta_j]), & \text{if } j > 0 \text{ and } k > \eta_j \end{cases} \quad (9)$$

Again, we can use formula for f_c in Equation (4) with Equation (7) to get the following:

$$\theta_c[m, n, p] = \max_{0 \leq j \leq n} f_c[m, j, \eta_j] \quad (10)$$

Algorithm 3 is the result of this optimization. We use the same notations used in Algorithm 2. We calculate $f_c[i, j, k]$ and $\theta_c[i, j, k]$ only where $k \in [\mu_j.. \eta_j]$. Line 6 calculates μ and η following Definitions 3 and 4. Therefore, if $x_i = y_j$ for any j , then $\mu_j = \mu$ and $\eta_j = \eta$. $\mu = 0$ means there is no such j and so Line 8 continues to next iteration of i , following fourth case of Equation (4). In Lines 13–17, we get the update rule for $T[k]$ by calculating $\theta_c[i-1, j, k]$ using the formula in Corollary 4. For Lines 18–20, the third case of Equation (4) is applicable, as $\mu = \mu_j$ and $\eta = \eta_j$ when $x_i = y_j$. The final statement in Line 21 follows from Equation (10).

This implementation runs in $O(mnc_p)$ time. Here, mn pairs of elements from X and Y are generated and for each such pair, at most c_p time is required to update the arrays F and T . The space complexity is $O(np)$. This however can be easily improved to $O(nc_p)$.

Fig. 3a shows an illustration of the reducing search space implementation considering the same input of Fig. 2a, i.e., $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$. Here, we show how T (Table b) is calculated using F (Table a) and then that T is used to calculate new F (Table c). Here, not all cells of F are computed and hence not all cells in T are needed. Interestingly, we can further improve the running time of our algorithm as we need only consider green cells of T . To achieve this, in the next section, we will use another quantity from which T can be easily computed.

4.4. An improved implementation: deferred computation

In the last implementation, for each i, j pair, we computed $\theta_c[i, j, k]$ (in array T) for all $k \in [\mu.. \eta]$. To do that, we used the $f_c[i, j, k]$ values (in matrix F) where $k \in [\mu_j.. \eta_j] \cap [\mu.. \eta]$. It means, we did $|\mu.. \eta|$ writes for $|\mu_j.. \eta_j| \cap |\mu.. \eta|$ reads. As no. of writes $\geq$ no. of reads, we want no. of writes to be equal to no. of reads. For this, we will defer the computation of θ_c until we get $x_i = y_j$. To aid that process, we introduce a new term $\tilde{\theta}_c$ from which θ_c can be computed. It is defined as follows:

$$\tilde{\theta}_c[i, j, k] = \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t)} f_c[i, t, k]. \quad (11)$$

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0	0	0	0	1	1
2	2	0	0	0	0	0	0
3	3	0	0	0	1	1	1
4	1	0	1	1	2	2	2
5	2	0	0	0	0	0	0
6	5	0	0	0	0	0	3
7	3	0	0	0	1	1	1
8	1	0	1	1	2	2	2

(a) $F[j, k] = f_c[i, j, k]$ at then end of i th iteration when $i = 4$ [$x_4 = 5$]

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0	0	0	0	1	1
2	2	0	0	0	0	1	1
3	3	0	0	0	1	1	1
4	1	0	1	1	2	2	2
5	2	0	1	1	2	2	2
6	5	0	1	1	2	2	3
7	3	0	1	1	2	2	3
8	1	0	1	1	2	2	3

(b) $T[k] = \theta_c[i - 1, j, k]$ at i th iteration, j th update when $i = 5$

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0	0	0	0	1	1
2	2	0	0	1	1	2	2
3	3	0	0	0	1	1	1
4	1	0	1	1	2	2	2
5	2	0	0	2	3	3	3
6	5	0	0	0	0	0	3
7	3	0	0	0	1	1	1
8	1	0	1	1	2	2	2

(c) $F[j, k] = f_c[i, j, k]$ at the end of i th iteration when $i = 5$ [$x_5 = 2$]

Fig. 2. State of table F and array T while calculating LCAIS by Algorithm 2 considering $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$. T (Table b) is calculated using F (Table a) and then that T is used to calculate new F (Table c). Here, red cell represents $z_k < y_j$, green cell represents $y_j \leq z_k < y_j + c$ and blue cell represents $z_k \geq y_j + c$.

Now, θ_c can be computed from $\tilde{\theta}_c$ using the following theorem.

Theorem 4.

$$\theta_c[i, j, k] = \max(\theta_c[i, j, k - 1], \tilde{\theta}_c[i, j, k]) \text{ when } k > 1. \quad (12)$$

Proof. From Equation (4),

$$f_c[i, j, k] = 0 = f_c[i, j, k - 1], \text{ if } j = 0 \text{ or } k < \mu_j \quad (13)$$

and

$$f_c[i, j, k] = f_c[i, j, \eta_j] = f_c[i, j, k - 1], \text{ if } j > 0 \text{ and } k > \eta_j \quad (14)$$

(as $f_c[i, j, k - 1]$ and $f_c[i, j, \eta_j]$ are: the same if $k - 1 = \eta_j$ and equal if $k - 1 > \eta_j$)

Algorithm 3 LCAIS: tracking the last element - reducing search space.

```

1: function LCAIS( $X, Y, c$ )
2:    $Z \leftarrow \text{MSEQ}(X, Y)$ 
3:    $M \leftarrow \text{INDEXMAP}(Z, Y), H \leftarrow \text{UPPERINDEXMAP}(Z, Y, c)$ 
4:   Initialize a matrix  $F$  of dimension  $(n+1) \times p$  with 0
5:   for  $i \leftarrow 1$  to  $m$  do
6:      $\mu \leftarrow \mu_Z(x_i), \eta \leftarrow \eta_{Z,c}(x_i)$  ▷ calculate  $\mu_Z, \eta_{Z,c}$  as described in Definitions 3, 4
7:     if  $\mu = 0$  then
8:       continue
9:     Initialize an array  $T$  of size  $p$  with 0
10:    for  $j \leftarrow 1$  to  $n$  do
11:      if  $x_i = y_j$  then
12:         $T' \leftarrow T$ 
13:      for  $k \leftarrow \mu$  to  $\eta$  do
14:        if  $k > \eta_j$  then
15:           $T[k] \leftarrow \max(T[k], F[j, \eta_j])$ 
16:        else if  $k \geq \mu_j$  then
17:           $T[k] \leftarrow \max(T[k], F[j, k])$ 
18:      if  $x_i = y_j$  then
19:        for  $k \leftarrow \mu$  to  $\eta$  do
20:           $F[j, k] \leftarrow T'[k] + 1$ 
21:  return  $\max_{0 \leq j \leq n} F[j, \eta_j]$ 

```

Now, from Equation (12), when $k > 1$,

$$\begin{aligned}
R.H.S. &= \max(\theta_c[i, j, k-1], \tilde{\theta}_c[i, j, k]) \\
&= \max(\max_{0 \leq t \leq j} f_c[i, t, k-1], \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t)} f_c[i, t, k]) \\
&\quad \text{[from Theorem 3 and Equation (11)]} \\
&= \max(\max_{t=0 \vee (1 \leq t \leq j \wedge k < \mu_t)} f_c[i, t, k-1], \max_{t=0 \vee (1 \leq t \leq j \wedge k > \eta_t)} f_c[i, t, k-1], \\
&\quad \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t)} f_c[i, t, k-1], \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t)} f_c[i, t, k]) \\
&\quad \text{[dividing first set into three nonempty sets that cover it]} \\
&= \max(\max_{t=0 \vee (1 \leq t \leq j \wedge k < \mu_t)} f_c[i, t, k], \max_{t=0 \vee (1 \leq t \leq j \wedge k > \eta_t)} f_c[i, t, k], \\
&\quad \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t)} f_c[i, t, k]) \quad \text{[using Equations (13), (14) on the first} \\
&\quad \text{two sets respectively and Corollary 1 on the last two sets]} \\
&= \max_{0 \leq t \leq j} f_c[i, t, k] \\
&= \theta_c[i, j, k] \quad \text{[from Theorem 3]} \\
&= L.H.S.
\end{aligned}$$

The proof is complete $\square$

Rather than computing θ_c from $\tilde{\theta}_c$ and then using it to compute f_c , we can directly compute f_c from $\tilde{\theta}_c$ as follows. Note that, the following theorem does not cover the case: $x_i = y_j$ and $k = \mu_j$. 以下定理不適用 有配對且位於Increasing

Theorem 5. If $x_i = y_j$ and $\mu_j < k \leq \eta_j$, then

$$f_c[i, j, k] = \max(f_c[i, j, k-1], \tilde{\theta}_c[i-1, j-1, k] + 1)$$

Proof. Using Equations (4) and (12) we have:

$$\begin{aligned}
f_c[i, j, k] &= \theta_c[i-1, j-1, k] + 1 \\
&= \max(\theta_c[i-1, j-1, k-1], \tilde{\theta}_c[i-1, j-1, k]) + 1 \\
&= \max(\theta_c[i-1, j-1, k-1] + 1, \tilde{\theta}_c[i-1, j-1, k] + 1) \\
&= \max(f_c[i, j, k-1], \tilde{\theta}_c[i-1, j-1, k] + 1) \\
&\quad \text{[using the third case of Equation (4), as } \mu_j \leq k-1 < \eta_j \text{]} \quad \square
\end{aligned}$$

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0				1	1
2	2	0		0	0	0	
3	3	0			1	1	1
4	1	0	1	1	2		
5	2	0		0	0	0	
6	5	0					3
7	3	0			1	1	1
8	1	0	1	1	2		

(a) $F[j, k] = f_c[i, j, k]$ at then end of i th iteration when $i = 4$ [$x_4 = 5$]

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j			0	0	0	
1	4			0	0	1	
2	2			0	0	1	
3	3			0	1	1	
4	1			1	2	2	
5	2			1	2	2	
6	5			1	2	2	
7	3			1	2	2	
8	1			1	2	2	

(b) $T[k] = \theta_c[i - 1, j, k]$ at i th iteration, j th update when $i = 5$ [$x_5 = 2$]

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0				1	1
2	2	0		1	1	2	
3	3	0			1	1	1
4	1	0	1	1	2		
5	2	0		2	3	3	
6	5	0					3
7	3	0			1	1	1
8	1	0	1	1	2		

(c) $F[j, k] = f_c[i, j, k]$ at then end of i th iteration when $i = 5$ [$x_5 = 2$]

Fig. 3. State of table F and array T while calculating LCAIS by Algorithm 3 considering $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$. T (Table b) is calculated using F (Table a) and then that T is used to calculate new F (Table c). Here, red cell represents $z_k < y_j$, green cell represents $y_j \leq z_k < y_j + c$, blue cell represents $z_k \geq y_j + c$. Empty cells represent values that we do not compute.

For implementation purposes, we need the following recursive definition of $\tilde{\theta}_c$ which follows readily from its original definition.

$$\tilde{\theta}_c[i, j, k] = \begin{cases} 0, & \text{if } j = 0, \\ \tilde{\theta}_c[i, j - 1, k], & \text{if } j > 0 \text{ and } (k < \mu_j \text{ or } k > \eta_j) \\ \max(\tilde{\theta}_c[i, j - 1, k], f_c[i, j, k]), & \text{if } j > 0 \text{ and } \mu_j \leq k \leq \eta_j \end{cases} \quad (15)$$

Algorithm 4 follows from Algorithm 3 after applying this optimization. Lines 2-8, 26 stay the same. T_μ represents $\theta_c[i - 1, j - 1, \mu]$ and $\tilde{T}[k]$ represents $\tilde{\theta}_c[i - 1, j - 1, k]$ for $k \in [\mu + 1.. \eta]$, at the start of iteration i, j . T'_μ and $\tilde{T}'$ are copies of T_μ and $\tilde{T}$, and do not change later in the loop. Hence, $T'_\mu = \theta_c[i - 1, j - 1, \mu]$ and $\tilde{T}'[k] = \tilde{\theta}_c[i - 1, j - 1, k]$ for $k \in [\mu + 1.. \eta]$. In Line 15, we can continue to next j , as $\mu_j = 0$ implies that no element in X is equal to y_j (i.e., case (4) of Equation (4)). Lines 16-19 update T_μ to represent $\theta_c[i - 1, j, \mu]$ using Corollary 4. Then, in Lines 20-21, $\tilde{T}'$ gets updated following Equation (15) so that $\tilde{T}'[k] = \tilde{\theta}_c[i - 1, j, k]$ for $k \in [\mu + 1.. \eta]$. Finally, in Line 23, $F[j, \mu]$ gets updated to $f_c[i, j, \mu]$ using the third case of Equation (4), and in Line 24-25, for each $k \in [\mu + 1.. \eta]$, $F[j, k]$ gets updated to $f_c[i, j, k]$ using Theorem 5.

Algorithm 4 LCAIS: tracking the last element - deferred computation.

```

1: function LCAIS( $X, Y, c$ )
2:    $Z \leftarrow \text{MSEQ}(X, Y)$ 
3:    $M \leftarrow \text{INDEXMAP}(Z, Y), H \leftarrow \text{UPPERINDEXMAP}(Z, Y, c)$ 
4:   Initialize a matrix  $F$  of dimension  $(n+1) \times p$  with 0
5:   for  $i \leftarrow 1$  to  $m$  do
6:      $\mu \leftarrow \mu_Z(x_i), \eta \leftarrow \eta_{Z,c}(x_i)$ 
7:     if  $\mu = 0$  then
8:       continue
9:      $T_\mu \leftarrow 0$ 
10:    Initialize an array  $\tilde{T}$  of size  $p$  with 0
11:    for  $j \leftarrow 1$  to  $n$  do
12:      if  $x_i = y_j$  then
13:         $T_{\mu'} \leftarrow T_\mu, \tilde{T}' \leftarrow \tilde{T}$ 
14:      else if  $\mu_j = 0$  then
15:        continue
16:      if  $\mu > \eta_j$  then
17:         $T_\mu \leftarrow \max(T_\mu, F[j, \eta_j])$ 
18:      else if  $\mu \geq \mu_j$  then
19:         $T_\mu \leftarrow \max(T_\mu, F[j, k])$ 
20:      for  $k \in [\mu + 1, \eta] \cap [\mu_j, \eta_j]$  do
21:         $\tilde{T}[k] \leftarrow \max(\tilde{T}[k], F[j, k])$ 
22:      if  $x_i = y_j$  then
23:         $F[j, \mu] \leftarrow T_{\mu'} + 1$ 
24:        for  $k \leftarrow \mu + 1$  to  $\eta$  do
25:           $F[j, k] \leftarrow \max(F[j, k-1], \tilde{T}'[k] + 1)$ 
26:   return  $\max_{0 \leq j \leq n} F[j, \eta_j]$ 

```

This implementation runs in $O(m(n + c_n c_p))$ time. Here, mn pairs of elements from X and Y are compared. For each element x_i , the loop in Lines 20–21 is active only if $y_j \leq x_i < y_j + c$ and runs in $O(c_p)$ time. Hence, by definition of c_n , it is active at most c_n times. Overall, this loop runs in $O(mc_n c_p)$ time. Again, the loop in Lines 24–25 is active when $x_i = y_j$ and runs in $O(c_p)$ time. As a result, this loop runs in $O(\mathcal{M}c_p)$ time overall. Also, it is easy to see that $mc_n c_p \geq \mathcal{M}c_p$. The space complexity is $O(np)$ which can easily be improved to $O(nc_p)$.

In Fig. 4a, we show the states of the tables when running this algorithm for $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$. Here, $\tilde{T}$ and T_μ (Table b) are calculated using F (Table a) and then they are used to calculate new F (Table c).

4.5. An improved implementation: using a priority search tree

So far we have not used any sophisticated data structure in our implementations. In this section, we discuss another implementation where we leverage a **balanced binary tree** type data structure. It has two functions:

TreeInsert($\mathcal{T}, k, v$): This function inserts a node with key k and value v into the tree $\mathcal{T}$. If the tree already has a node with key k , then that node gets updated.

TreeFind($\mathcal{T}, v$): This function returns the maximum key in tree $\mathcal{T}$ such that the corresponding value is at most k .

This data structure can be implemented using a Priority Search Tree [22] which gives us $O(\log n)$ time complexity for both operations using $O(n)$ space. Here, n is the number of elements the tree holds. We use this tree data structure as follows. For some i, k a tree $\mathcal{T}_k^i$ holds a set of key-value pairs $(f_c[i, j, k], j)$ such that $\mu_j \leq k \leq \eta_j$. So, we can see the tree as a set of pairs,

$$\mathcal{T}_k^i = \{(f_c[i, j, k], j) \mid 0 \leq j \leq m, \mu_j \leq k \leq \eta_j\}$$

Now, when **TreeFind**($\mathcal{T}_k^i, j$) is called, it returns maximum of $f_c[i, t, k]$ such that $t \leq j$ and $\mathcal{T}_k^i$ contains $(f_c[i, t, k], t)$. Equivalently,

$$\begin{aligned} \text{TreeFind}(\mathcal{T}_k^i, j) &= \max\{f_c[i, t, k] \mid 0 \leq t \leq j, \mu_t \leq k \leq \eta_t\} \\ &= \tilde{\theta}_c[i, j, k], \text{ [using Equation (11)]} \end{aligned} \tag{16}$$

Before getting into the implementation details, we need to redefine $\tilde{\theta}_c$ that makes our work easy with this data-structure. We prove the following corollary.

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0				1	1
2	2	0		0	0	0	
3	3	0			1	1	1
4	1	0	1	1	2		
5	2	0		0	0	0	
6	5	0					3
7	3	0			1	1	1
8	1	0	1	1	2		

(a) $F[j, k] = f_c[i, j, k]$ at then end of i th iteration when $i = 4$ [$x_4 = 5$]

j	T_μ	$\tilde{T}[3]$	$\tilde{T}[4]$
0	y_j	0	0
1	4	0	1
2	2	0	1
3	3	1	1
4	1	2	1
5	2	2	1
6	5	2	1
7	3	2	1
8	1	2	1

(b) $T_\mu = \theta_c[i - 1, j, \mu]$, $\tilde{T}[k] = \tilde{\theta}_c[i - 1, j, k]$ at i th iteration, j th update when $i = 5$ [$x_5 = 2$]. Here, $\mu = 2$

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0				1	1
2	2	0		1	1	2	
3	3	0			1	1	1
4	1	0	1	1	2		
5	2	0		2	3	3	
6	5	0					3
7	3	0			1	1	1
8	1	0	1	1	2		

(c) $F[j, k] = f_c[i, j, k]$ at then end of i th iteration when $i = 5$ [$x_5 = 2$]

Fig. 4. State of F , $\tilde{T}$ and T_μ while calculating LCAIS by Algorithm 4 considering $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$. $\tilde{T}$ and T_μ (Table b) are calculated using F (Table a) and then they are used to calculate new F (Table c). Here, red cell represents $z_k < y_j$, green cell represents $y_j \leq z_k < y_j + c$, blue cell represents $z_k \geq y_j + c$. Empty cells represent values that we do not compute.

Corollary 5. $\tilde{\theta}_c[i, j, k]$ can be redefined as

$$\tilde{\theta}_c[i, j, k] = \begin{cases} 0, & \text{if } i = 0 \\ \max(\tilde{\theta}_c[i - 1, j, k], \bar{\theta}_c[i, j, k]), & \text{if } i > 0 \end{cases} \quad (17)$$

where

$$\bar{\theta}_c[i, j, k] = \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t \wedge x_i = y_t)} f_c[i, t, k]. \quad (18)$$

Proof. From Equation (11), it is easy to see that if $i = 0$, $\tilde{\theta}_c[i, j, k] = 0$. Again, from Equation (11), we can write, if $i > 0$,

$$\begin{aligned} L.H.S. = \tilde{\theta}_c[i, j, k] &= \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t)} f_c[i, t, k] \\ &= \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t \wedge x_i \neq y_t)} f_c[i, t, k], \end{aligned}$$

$$\begin{aligned}
& \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t \wedge x_i = y_t)} f_c[i, t, k]) \\
&= \max(\max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t \wedge x_i \neq y_t)} f_c[i-1, t, k], \\
& \quad \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t \wedge x_i = y_t)} f_c[i-1, t, k], \\
& \quad \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t \wedge x_i = y_t)} f_c[i, t, k]) \\
& \quad \text{[using Equation (4) on first set, splitting the second set following Corollary 2]} \\
&= \max(\max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t)} f_c[i-1, t, k], \max_{t=0 \vee (1 \leq t \leq j \wedge \mu_t \leq k \leq \eta_t \wedge x_i = y_t)} f_c[i, t, k]) \\
& \quad \text{[combining first two sets]} \\
&= \max(\tilde{\theta}_c[i-1, j, k], \bar{\theta}_c[i, j, k]) \text{ [using Equation (11) and (18)]} \\
&= R.H.S
\end{aligned}$$

The proof is complete. $\square$

We use the tree operations on $\mathcal{T}$ in place of array $\tilde{T}$ in Algorithm 4 to get Algorithm 5. Here, at the start of iteration i , $\mathcal{T}[k]$ represents $\mathcal{T}_k^{i-1}$. In Line 23, for each $k \in [\mu + 1.. \eta]$, $F[j, k]$ gets updated to $f_c[i, j, k]$ using Equation (16) and subsequently Theorem 5. Then, in Line 27, for each $k \in [\mu.. \eta]$, $\mathcal{T}[k]$ gets updated to $\mathcal{T}_k^i$ using Equation (17).

Algorithm 5 LCAIS: tracking the last element - using priority search tree.

```

1: function LCAIS( $X, Y, c$ )
2:    $Z \leftarrow \text{MSEQ}(X, Y)$ 
3:    $M \leftarrow \text{INDEXMAP}(Z, Y)$ ,  $H \leftarrow \text{UPPERINDEXMAP}(Z, Y, c)$ 
4:   Initialize a matrix  $F$  of dimension  $(n+1) \times p$  with 0
5:   Initialize a list of  $p$  priority search trees  $\mathcal{T}$  with  $(0, 0)$ 
6:   for  $i \leftarrow 1$  to  $m$  do
7:      $\mu \leftarrow \mu_Z(x_i)$ ,  $\eta \leftarrow \eta_{Z,c}(x_i)$ 
8:     if  $\mu = 0$  then
9:       continue
10:     $T_\mu \leftarrow 0$ 
11:    for  $j \leftarrow 1$  to  $n$  do
12:      if  $x_i = y_j$  then
13:         $T_\mu' \leftarrow T_\mu$ 
14:      else if  $\mu_j = 0$  then
15:        continue
16:      if  $\mu > \eta_j$  then
17:         $T_\mu \leftarrow \max(T_\mu, F[j, \eta_j])$ 
18:      else if  $\mu \geq \mu_j$  then
19:         $T_\mu \leftarrow \max(T_\mu, F[j, \mu])$ 
20:      if  $x_i = y_j$  then
21:         $F[j, \mu] \leftarrow T_\mu' + 1$ 
22:        for  $k \leftarrow \mu + 1$  to  $\eta$  do
23:           $F[j, k] \leftarrow \max(F[j, k-1], \text{TreeFind}(\mathcal{T}[k], j-1) + 1)$ 
24:      for  $k \leftarrow \mu$  to  $\eta$  do
25:        for  $j \leftarrow 1$  to  $n$  do
26:          if  $x_i = y_j$  then
27:             $\mathcal{T}[k] \leftarrow \text{TreeInsert}(\mathcal{T}[k], F[j, k], j)$ 
28:   return  $\max_{0 \leq j \leq n} F[j, \eta_j]$ 

```

This implementation runs in $O(mn + \mathcal{M}c_p \log \ell)$ time if we use the Priority Search Tree of [22]. Here, nm pairs of elements from X and Y are generated and for $\mathcal{M}$ matches, at most c_p tree operations are required to update the array F . Each tree-operation takes $O(\log \ell)$ time (as each tree holds at most ℓ elements). The space complexity is $O(np)$. Here, $O(np)$ space is used for F , and $O(\ell p)$ is used for T (as there are p trees in T and each tree contains at most ℓ nodes).

The $O(\log \ell)$ tree-operation time can be changed to $O(\log \log E)$, where E is the maximum number of elements that can be stored in the tree, using a dynamic predecessor data structure (such as a Van Emde Boas Tree [10]). The idea would be to maintain prefix maxima only in a predecessor data structure (these maxima are simultaneously sorted by both coordinates). A prefix maximum query then reduces to a single predecessor query. At insertion time, a single predecessor query could be used to decide whether the new element is a prefix maximum (i.e., if it is not dominated by any existing prefix maximum). If it is, then the prefix maximum dominated by the new element should be deleted. While this costs $O(\log \log E)$ time per element deleted, the amortized complexity of insertion remains $O(\log \log E)$, and the amortization does not affect the

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0				1	1
2	2	0		0	0	0	
3	3	0			1	1	1
4	1	0	1	1	2		
5	2	0		0	0	0	
6	5	0					3
7	3	0			1	1	1
8	1	0	1	1	2		

(a) $F[j, k] = f_c[i, j, k]$ at then end of i th iteration when $i = 4$ [$x_4 = 5$]

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0	0	0	0	1	1
2	2	0	0	0	0	1	1
3	3	0	0	0	1	1	1
4	1	0	1	1	2	1	1
5	2	0	1	1	2	1	1
6	5	0	1	1	2	1	3
7	3	0	1	1	2	1	3
8	1	0	1	1	2	1	3

(b) $\text{TreeFind}(\mathcal{T}[k], j-1) = \tilde{\theta}_c[i-1, j, k]$ at i th iteration, j th update when $i = 5$

	k	0	1	2	3	4	5
j	z_k	1	2	3	4	5	
0	y_j	0	0	0	0	0	0
1	4	0				1	1
2	2	0		1	1	2	
3	3	0			1	1	1
4	1	0	1	1	2		
5	2	0		2	3	3	
6	5	0					3
7	3	0			1	1	1
8	1	0	1	1	2		

(c) $F[j, k] = f_c[i, j, k]$ at then end of i th iteration when $i = 5$ [$x_5 = 2$]

Fig. 5. State of table F and list of trees $\mathcal{T}$ while calculating LCAIS by Algorithm 5. $\text{TreeFind}(\mathcal{T}[k], j-1)$ (Table b) is computed from F (Table a) and then used to calculate new F (Table c). Here, in Table a and Table c, red cell represents $z_k < y_j$, green cell represents $y_j \leq z_k < y_j + c$, blue cell represents $z_k \geq y_j + c$, and red numbers represent TreeInsert operations. In Table b, gray cells represent computation of $\text{TreeFind}(\mathcal{T}[k], j)$, other cells are not computed and red numbers are what the trees actually store.

worst-case complexity of the resulting LCAIS algorithm. Using this data-structure requires $O(E)$ additional space, implying $O(np + Ep)$ space for Algorithm 4. Our algorithm requires $E = \min(m, n)$.

In Fig. 5a, we show the states of the tables when this algorithm is run on $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$. Here, $\text{TreeFind}(\mathcal{T}[k], j-1)$ (Table b) is computed from F (Table a) and then used to calculate new F (Table c).

5. Approach 3: track the index of lengths

If we carefully scrutinize Table b of Fig. 1a, we can see some particular patterns as follows (formal proofs follow).

- In each column, its values start from zero at the top and going down, the value either increases by one or stays the same. It is very common in problems related to LCS.
- When the value goes up, it is either in green or blue cell.
- Each blue cell is the maximum of its top and left cell.

$$i = 5$$

$$x_i = 2$$

	k	0	1	2	3	4	5
l	z_k	1	2	3	4	5	
0		0	0	0	0	0	0
1			4	2	2	1	1
2				5	4	2	2
3					5	5	5

Fig. 6. A table corresponding to Table b of Fig. 1a with length l on rows and k values on columns. Cell $[l, k]$ contains the minimum j index for which $\theta_c[i, j, k] = l$ on the k^{th} column of Table b in Fig. 1a. Here, red cell represents $z_k < x_i$, green cell represents $x_i \leq z_k < x_i + c$ and blue cell represents $z_k \geq x_i + c$. Empty cell means no j index has length l on the k^{th} column of Table b in Fig. 1a.

This gives us the idea to follow only the topmost index (i.e., the minimum j index) of a θ_c value. In Fig. 6, we generate a table corresponding to Table b of Fig. 1a where cell $[l, k]$ contains the minimum j index for which $\theta_c[i, j, k] = l$ on the k^{th} column of Table b in Fig. 1a.

Again, we can see some particular patterns:

- Each row is non-increasing from left to right.
- Each column is increasing from top to bottom.

In the next section, we will formalize these patterns to derive a recursive formula. Before we proceed, recall that μ_j and η_j are shorthands for $\mu_Z(y_j)$ and $\eta_{Z,c}(y_j)$ respectively. Similarly, μ_i and η_i are shorthands for $\mu_Z(x_i)$ and $\eta_{Z,c}(x_i)$ respectively.

5.1. Recursive formula

Theorem 6. Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ and $Z = \langle z_1, z_2, \dots, z_p \rangle$ is their MSeq. If we define $\sigma_c[i, k, l]$ to be the minimum index $j \in [0..n]$ such that there exists a CAIS of X_i and Y_j of length l that has maximum element less than or equal to z_k and $\sigma_c[i, k, l] = \infty$ when no such j exists, then $\sigma_c[i, k, l]$ can be computed by the following dynamic programming formulation.

$$\sigma_c[i, k, l] = \begin{cases} 0, & \text{if } l = 0, \\ \infty, & \text{if } l > 0 \text{ and } i = 0, \\ \sigma_c[i - 1, k, l], & \text{if } l, i > 0 \text{ and } k < \mu_i, \\ \min(\sigma_c[i - 1, k, l], \sigma_c[i, k - 1, l]), & \text{if } l, i > 0 \text{ and } k > \eta_i, \\ \min(\sigma_c[i - 1, k, l], \lambda_c[i, k, l]), & \text{if } l, i > 0 \text{ and } \mu_i \leq k \leq \eta_i. \end{cases} \quad (19)$$

Here,

$$\lambda_c[i, k, l] = \min(\{\infty\} \cup \{j \mid 1 \leq j \leq n, x_i = y_j, \sigma_c[i - 1, k, l - 1] < j\}) \quad (20)$$

Proof. (1) It is trivial that $\sigma_c[i, k, l] = 0$ when $l = 0$.

(2) It is impossible to get a CAIS of length $l > 0$ when one of the sequences is empty. As X_0 is empty, by definition $\sigma_c[i, k, l] = \infty$ when $i = 0$.

For the remaining cases, $l, i > 0$.

Let $\sigma_c[i, k, l] = j$ and if $j \neq \infty$, then the corresponding sequence, $S = \langle s_1, s_2, \dots, s_l \rangle$ is a CAIS of X_i and Y_j of length l such that $\max(S) \leq z_k$.

Here, $j = \infty$ implies no prefix of Y satisfies the constraints for X_i . Now, removing an element from X_i does not make the situation any better. So, no prefix of Y satisfies the constraints for X_{i-1} , if $j = \infty$. More formally,

$$\sigma_c[i - 1, k, l] = j, \text{ when } j = \infty \quad (21)$$

Again, if $j \neq \infty$, then $s_l \neq x_i$ implies S is a CAIS of X_{i-1} and Y_j of length l such that $\max(S) \leq z_k$. Hence,

$$\sigma_c[i - 1, k, l] = j, \text{ when } j \neq \infty \text{ and } s_l \neq x_i \quad (22)$$

(3) If $k < \mu_i$, then $z_k < z_{\mu_i} \leq x_i$ (by Lemma 1); which implies a CAIS of X_i and Y_j having maximum element at most z_k cannot contain x_i . So, if $j \neq \infty$, then $s_l \neq x_i$ which implies equation (22). Hence, Equations (21), (22) imply,

$$\sigma_c[i, k, l] = \sigma_c[i - 1, k, l].$$

For the remaining cases, we need the following inequality,

$$\sigma_c[i-1, k, l] \geq j, \text{ when } j \neq \infty \quad (23)$$

For contradiction, suppose $\sigma_c[i-1, k, l] = j' < j$ and S' is a corresponding sequence. Now, S' is a valid CAIS of X_i and $Y_{j'}$ of length l and $\max(S') \leq z_k$. Hence, $\sigma_c[i, k, l] = j \leq j'$ which contradicts our assumption that $j > j'$ proving Equation (23).

(4) If $k > \eta_i$, then $z_k > z_{\eta_i}$.

Now, when $j = \infty$, making the maximum element smaller does not make the situation any better. So, no prefix of Y satisfies the constraints for z_{k-1} . Hence,

$$\sigma_c[i, k-1, l] = j, \text{ when } j = \infty \quad (24)$$

Now, for the case $j \neq \infty$, we state and prove the following equation.

$$\sigma_c[i, k-1, l] \geq j, \text{ when } j \neq \infty \quad (25)$$

We prove Equation (25) by contradiction. For the sake of contradiction, suppose $\sigma_c[i, k-1, l] = j' < j$ and S' is a corresponding sequence. Now, S' is a valid CAIS of X_i and $Y_{j'}$ of length l and $\max(S') \leq z_{k-1} < z_k$. Hence, $\sigma_c[i, k, l] = j \leq j'$ which contradicts our assumption that $j > j'$ proving Equation (25).

However, if $s_l = x_i$, then $\max(S) \leq z_{\eta_i}$ (by Lemma 5) $< z_k$. So, $\max(S) \leq z_{k-1}$, which means that S is a CAIS of X_i and Y_j of length l such that $\max(S) \leq z_{k-1}$. Hence,

$$\sigma_c[i, k-1, l] = j, \text{ when } j \neq \infty \text{ and } s_l = x_i \quad (26)$$

Equations (21), (22), (23), (24), (25), (26) imply,

$$\sigma_c[i, k, l] = \min(\sigma_c[i-1, k, l], \sigma_c[i, k-1, l]).$$

(5) If $\mu_i \leq k \leq \eta_i$, then $z_k \geq z_{\mu_i} = x_i$ and $z_k \leq z_{\eta_i} < x_i + c$ (by Lemma 3).

Now, we need to prove that if $j = \infty$, then $\lambda_c[i, k, l] = \infty$. We prove this by contradiction as follows. Assume for the sake of contradiction that $\lambda_c[i, k, l] = j'$. Then $x_i = y_{j'}$ and $\sigma_c[i-1, k, l-1] = j'' < j'$. So, there exists a sequence S' that is a CAIS of X_{i-1} and $Y_{j''}$ of length $(l-1)$ such that $\max(S') \leq z_k$. Now, appending $x_i = y_{j'}$ to S' produces a CAIS of length l that has maximum element less than or equal to $\max(x_i, z_k) = z_k$ ($z_k \geq z_{\mu_i} = x_i$ by Lemma 2). It contradicts to our assumption that $j = \infty$ (i.e., no CAIS of length l exists that satisfies the constraints). Hence,

$$\lambda_c[i, k, l] = j, \text{ when } j = \infty \quad (27)$$

Again, for the case $j \neq \infty$, we can say,

$$\lambda_c[i, k, l] \geq j, \text{ when } j \neq \infty \quad (28)$$

For contradiction, suppose $\lambda_c[i, k, l] = j' < j$. It means, $x_i = y_{j'}$ and $\sigma_c[i-1, k, l-1] < j'$ (Suppose S' is a corresponding sequence). So, $\max(S') \leq z_k$. Now, $S' \cdot x_i$ is a valid CAIS of X_i and $Y_{j'}$ of length l and $\max(S' \cdot x_i) \leq \max(z_k, x_i) = z_k$. Hence, $\sigma_c[i, k, l] = j \leq j'$ which contradicts our assumption that $j > j'$ proving Equation (25).

However, if $s_l = x_i$, then there exists $j' \in [1..n]$ such that $x_i = y_{j'}$ and S_{l-1} is a CAIS of X_{i-1} and $Y_{j''}$ ($j'' < j'$) of length $(l-1)$ such that $\max(S_{l-1}) \leq z_k$. As $\sigma_c[i, k, l]$ is the minimum of such j' , we can write, $\sigma_c[i, k, l] = j = \min\{j' | 1 \leq j' \leq n, x_i = y_{j'}, \sigma_c[i-1, k, l-1] < j'\}$. Hence,

$$\lambda_c[i, k, l] = j, \text{ when } j \neq \infty \text{ and } s_l = x_i \quad (29)$$

Equations (21), (22), (23), (27), (28), (29) imply,

$$\sigma_c[i, k, l] = \min(\sigma_c[i-1, k, l], \lambda_c[i, k, l]).$$

The proof is complete. $\square$

From this formulation, we can give the following corollaries which will be useful to prove the correctness of our algorithm. Here, ℓ_i is the length of LCAIS of X_i (i^{th} prefix of X) and Y with parameter c . Note its difference with ℓ , which is the length of X (the whole sequence) and Y with parameter c .

Corollary 6. $\forall 1 \leq i \leq m, 2 \leq k \leq p, 1 \leq l \leq \ell_i : \sigma_c[i, k, l] \leq \sigma_c[i, k-1, l]$.

Corollary 7. $\forall 1 \leq i \leq m, 1 \leq k \leq p, 1 \leq l \leq \ell_i : \sigma_c[i, k, l] < \sigma_c[i, k, l-1]$.

Algorithm 6 LCAIS: tracking index of length.

```

1: function LCAIS( $X, Y, c$ )
2:    $Z \leftarrow \text{MSEQ}(X, Y)$ 
3:    $M \leftarrow \text{INDEXMAP}(Z, X), H \leftarrow \text{UPPERINDEXMAP}(Z, X, c)$ 
4:    $h \leftarrow 0$ 
5:   for  $k \leftarrow 1$  to  $p$  do
6:      $S[k, 0] \leftarrow 0$ 
7:      $S[k, 1] \leftarrow \infty$ 
8:   for  $i \leftarrow 1$  to  $m$  do
9:     for  $j \leftarrow n$  down to 1 do
10:      if  $x_i \neq y_j$  then
11:        continue
12:       $l \leftarrow 1$ 
13:      for  $k \leftarrow \mu_i$  to  $\eta_i$  do
14:        while  $S[k, l] < j$  do
15:           $l \leftarrow l + 1$ 
16:           $S[k, l] \leftarrow j$ 
17:          if  $l > h$  then
18:             $h \leftarrow l$ 
19:            for  $k' \leftarrow 1$  to  $p$  do
20:               $S[k', h + 1] \leftarrow \infty$ 
21:            for  $k \leftarrow \eta_i + 1$  to  $p$  do
22:              if  $S[k, l] \leq j$  then
23:                break
24:               $S[k, l] \leftarrow j$ 
25:   return  $h$ 

```

5.2. Implementation

Now let us get into the implementation details of this approach in Algorithm 6. Here h represents ℓ_i at the end of i^{th} iteration. Recall that ℓ_i represents the length of an LCAIS of X_i and Y . Initially $i = 0$, i.e., X_i is empty and hence $h = \ell_i = 0$ (Line 4). $S[k, l]$ represents $\sigma_c[i, k, l]$ at the end of the i^{th} iteration. As initially $i = 0$, the initialization (Line 5–7) follows from case (1) and (2) of Equation (19). The matrix S has dimension $p \times (h + 1)$ and throughout the implementation we ensure this dimension even though h changes.

For each iteration i , we iterate over j in a descending manner from n to 1. For each i, j pair, we check if $x_i \neq y_j$ (Line 10). If it does satisfy this check, we continue to next j (Line 11). The reason for this is that none of the cases of Equation (19) makes any changes from iteration $i - 1$ to i when $x_i \neq y_j$. Hence, the following lines (Lines 12–24) are applicable for $x_i = y_j$. As the cases (1), (2) and (3) of Equation (19) do not make any changes from iteration $i - 1$ to i , we only consider Cases (4) (Lines 21–24) and (5) (Lines 12–20).

Now, for handling case (5) of Equation (19), the idea is to update $S[k, l]$ with j for those index pairs k, l when we know $\sigma_c[i - 1, k, l] \geq j$ and $\sigma_c[i, k, l] \leq j$ but skip those k, l pairs for which we are sure that $\sigma_c[i, k, l] < j$ (later iterations of j will take care of it). Lines 12–16 ensure this.

If the loop at Lines 14–15 starts with a value of l such that $\sigma_c[i - 1, k, l - 1] < j$, then it ends with a value of l such that $\sigma_c[i - 1, k, l - 1] < j$ and $\sigma_c[i - 1, k, l] \geq j$ (by Corollary 7). Now, the condition $\sigma_c[i - 1, k, l - 1] < j$ holds at the initial iteration of loop in 13–20, because initially $l = 1$, which means $\sigma_c[i - 1, k, l - 1] = 0$ (from case (1) of Equation (19)). The condition $\sigma_c[i - 1, k, l - 1] < j$ holds for subsequent iterations, as $\sigma_c[i - 1, k - 1, l - 1] < j$ holds from previous iteration and it implies $\sigma_c[i - 1, k, l - 1] < j$ (by Corollary 6).

The condition $\sigma_c[i - 1, k, l - 1] < j$ and $\sigma_c[i - 1, k, l] \geq j$, that holds at the end of loop at Lines 14–15, implies that $\sigma_c[i, k, l] \leq j$ (from case (5) of Equation (19) and Equation (20)) but we cannot be sure if $\sigma_c[i, k, l] < j$. Thus we can do the update $S[k, l] \leftarrow j$ (Line 16). This update means that there is a CAIS of X_i and Y of length l .

Lines 17–18 ensure that $h (= \ell_i)$ represents the length of an LCAIS of X_i and Y . As h can only increase by 1 at a time, we append a column to matrix S to keep its dimension $p \times (h + 1)$ (Lines 19–20).

Lines 21–24 handle case (4) of Equation (19). At the end of loop at Lines 13–20, we end up with a value of l for which the condition $\sigma_c[i, \eta_i, l] \leq j$ holds (Line 16 ensures that). Now, using Corollary 6 we can show that $\forall \eta_i < k \leq p : \sigma_c[i, k, l] \leq j$. Hence we update $S[k, l]$ with j for $k \in [\eta_i + 1..p]$ (Lines 21–24). In Line 22–23, we can do this break because of Corollary 6.

This implementation runs in $O(mn + \mathcal{M}(p + \ell))$ time. Here, $O(p\ell)$ time is needed to generate S array; $O(mn)$ time is needed to do the nested loop for looking through each pair of element from X and Y ; and $O(\mathcal{M}(p + \ell))$ time is needed to find and update the elements in S array for the matching pairs from X and Y . Recall that $\mathcal{M}$ is the number of matches between X and Y . Each match causes $O(p)$ updates and takes $O(\ell)$ time to locate the update positions in S . The space complexity is $O(p\ell)$, as matrix S is of dimension $p \times (\ell + 1)$. Notably, for a sequence with no repeating elements (assumed in [23]), this implementation runs in $O(mn)$ time, a clear improvement over their algorithm.

Fig. 7a shows different states of the table S produced by this algorithm on $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$.

	k	0	1	2	3	4	5
l	z_k	1	2	3	4	5	
0	0	0	0	0	0	0	
1	∞	4	2	2	1	1	
2	∞	∞	5	4	2	2	
3	∞	∞	∞	5	5	5	

(a) $i = 6; x_i = 4$

	k	0	1	2	3	4	5
l	z_k	1	2	3	4	5	
0	0	0	0	0	0	0	
1	∞	4	2	2	1	1	
2	∞	∞	5	4	2	2	
3	∞	∞	∞	5	5	5	
4	∞	∞	∞	∞	∞	6	

(b) $i = 7; x_i = 5$

	k	0	1	2	3	4	5
l	z_k	1	2	3	4	5	
0	0	0	0	0	0	0	
1	∞	4	2	2	1	1	
2	∞	8	4	4	2	2	
3	∞	∞	8	5	5	5	
4	∞	∞	∞	8	8	6	

(c) $i = 8; x_i = 1$

Fig. 7. State of table S while calculating LCAIS by Algorithm 6 with $X = \langle 3, 1, 4, 5, 2, 4, 5, 1 \rangle$, $Y = \langle 4, 2, 3, 1, 2, 5, 3, 1 \rangle$ and $c = 3$. Here, red cell represents $z_k < x_i$, green cell represents $x_i \leq z_k < x_i + c$, blue cell represents $z_k \geq x_i + c$.

6. Recovering the output sequence

Up to this point we have only discussed how to calculate the length of an LCAIS. To recover an LCAIS, we can simply keep track of the length changes. But it would take cubic space. It is possible to make it more space efficient (quadratic space). We will discuss both a simple and an improved recovery methods for our second approach (Section 4). The methods can be easily extended for our third approach (Section 5). We only show the improved recovery method for this (Section 6.2.6). We do not consider our baseline algorithm (Section 3) for this purpose as it was mainly used for developing the subsequent algorithms.

6.1. A simple approach

There are various ways to keep track of the length changes. We can use separate variables to do it. But to make changes easy, we will store the extra information in the variables already being used in our implementations. In all the implementations of the second approach (Section 4), we keep integer values for lengths. Instead, now we will keep a triple, (ℓ, b, ptr) containing the following three pieces of information: (i) length (ℓ) (ii) last element in the sequence (b) and (iii) reference to the triple corresponding to the previous element in the sequence (ptr). This will form a tree where the triples are nodes with each node storing two data and a reference to the parent node. We will need the following changes to realize this. In what follows, for a node (i.e., triple) nd , we will conveniently use $nd[0]$, $nd[1]$ and $nd[2]$ to represent ℓ , b and ptr .

Initialization The lengths are initialized with 0 meaning empty sequences. So, the corresponding data are empty. Therefore, we will change all initialization to 0 with following triple:

$(0, nil, nil)$

max function As we replaced the integers with triples, we need to redefine our *max* function to be applied on two nodes (i.e., triples) as follows.

$$\max(node_1, node_2) = \begin{cases} node_1 & \text{if } node_1[0] \geq node_2[0] \\ node_2 & \text{otherwise} \end{cases}$$

Adding elements When a new element is added to a sequence, we increment its length. Now we will additionally keep the added element and its predecessor. Instead of incrementing the length, we will call the following function, where *node* is the predecessor node and *e* is the element being added.

function ADDELEMENT(*node*, *e*)
return ($node[0] + 1, e, node$)

6.1.1. Backtracking

After these changes are made, the resulting value from the algorithm is now a triple, which we call *maxNode*. We can easily do backtracking to recover the sequence as follows:

function BACKTRACK(*maxNode*)
 $S \leftarrow \{\}$
 $node \leftarrow maxNode$
while $node \neq nil$ **do**
 $S \leftarrow node[1] \cdot S$
 $node \leftarrow node[2]$
return S

6.1.2. Time and space complexity

The backtrack process stated above takes $O(l)$ time. It does not change the overall time complexity of the LCAIS algorithms. This method creates at most $O(c_p)$ triples for each matching pair. So, $O(\mathcal{M}c_p)$ triples are created in this process. Again, at least $O(nc_p)$ space is used to keep track of these triples (Algorithm 3 and 4). Hence, this process uses $O((\mathcal{M} + n)c_p)$ space. This is at most cubic. But for integer sequences ($c_p \leq c$) or when number of matching pairs is linear (e.g., when the elements are repeated at most constant number of times), this space is quadratic.

6.2. An improved approach

The simple method can be space efficient for some special cases. But it is still cubic for general cases. In the paper [26], they have showed a way to use $O(mn)$ space to recover an LCAIS. We can use the same idea here to get $O(mn)$ space recovery method. Before we get into the algorithm, we need to define some terminologies and the Specialized LCAIS Problem.

6.2.1. Additional terminologies

We follow the notations used in [26] after slightly adopting those to match our notations.

Definition 5 (Substring). For a given sequence $X = \langle x_1, x_2, \dots, x_m \rangle$, a substring of X from i^{th} to j^{th} element is denoted by $X_{i,j} = \langle x_i, x_{i+1}, \dots, x_j \rangle$, where $1 \leq i \leq j \leq m$.

Definition 6 (Representatives). For a given sequence $S = \langle s_1, s_2, \dots, s_l \rangle$, its representative, written as $\gamma(S)$, is the last maximum element in S . Hence, if $\gamma(S) = s_t$, then $\forall 1 \leq i < t : s_t \geq s_i$ and $\forall t < i \leq l : s_t > s_i$.

Definition 7 (A Representative Point). Suppose that $S = \langle s_1, s_2, \dots, s_l \rangle$ is an LCAIS of two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ for a given constant $c > 0$ and S occurs as $x_{i_1}, \dots, x_{i_l}$ and $y_{j_1}, \dots, y_{j_l}$. Now, if $\gamma(S) = s_t$, then (i_t, j_t) is a Representative Point.

6.2.2. Specialized LCAIS problem

Given two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ with $x_1 = y_1$, and a constant $c > 0$, a LCAIS of X and Y such that $(1, 1)$ is its representative point is called *Specialized LCAIS*.

Theorem 7. [26] Let $\delta[i, j]$ be the length of a Specialized LCAIS. $\delta[i, j]$ can be calculated by the following formula.

$$\delta[i, j] = \begin{cases} 0 & \text{if } i, j = 0 \\ \delta[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_{i-1} - c < x_i = y_j < x_1 \\ \max(\delta[i-1, j], \delta[i, j-1]) & \text{otherwise} \end{cases} \quad (30)$$

This formula is very similar to that of LCS. So, we can construct a *Specialized LCAIS* in $O(mn)$ time and space.

6.2.3. Required changes

In [26], the authors built an $m \times n$ table *Prev*. If $S = \langle s_1, \dots, s_t \rangle$ is an LCAIS of X_i and Y_j such that (i, j) and (p, q) are representative points of S and S_{t-1} respectively, then *Prev* $[i, j]$ stores the pair (p, q) . We already keep track of the representative, albeit a bit differently. To build the *Prev* table, we need to additionally keep track of the representative points. So, instead of keeping only the length, we will keep a triple, (ℓ, i, j) containing the following three pieces of information, namely, the length (ℓ), and the two indices (i, j) for a representative point corresponding to a sequence of that length.

We now have to do the same thing as we did in the simple approach. The *max* function is the same as in the simple approach. The initialization triple is now: $(0, 0, 0)$. The *ADDELEMENT* function is changed to following:

```

function ADDELEMENT(node, i, j)
  (l, p, q)  $\leftarrow$  node
  if  $x_i \geq x_p$  then
    Prev $[i, j] \leftarrow (p, q)$ 
    return (l + 1, i, j)
  else
    return (l + 1, p, q)

```

Note that, our implementation assumes that *ADDELEMENT* gets called to generate optimal subsequences only (i.e., *node* corresponds to an LCAIS of X_{i-1} and Y_{j-1} such that (p, q) is its representative point and adding $x_i = y_j$ to this sequence creates an LCAIS of X_i and Y_j such that its representative point is (i, j) if $x_i \geq x_p$ and (p, q) otherwise). This condition is true for all our algorithms (Algorithms 1-5).

Algorithm 7 LCAIS: tracking index of length with recovery.

```

1: function LCAIS( $X, Y, c$ )
2:    $Z \leftarrow \text{MSEQ}(X, Y)$ 
3:    $M \leftarrow \text{INDEXMAP}(Z, X), H \leftarrow \text{UPPERINDEXMAP}(Z, X, c)$ 
4:    $h \leftarrow 0$ 
5:   Initialize a matrix  $Prev$  of dimension  $(m+1) \times (n+1)$  with  $(0, 0)$ 
6:   for  $k \leftarrow 1$  to  $p$  do
7:      $S[k, 0] \leftarrow (0, 0)$ 
8:      $S[k, 1] \leftarrow (\infty, \infty)$ 
9:   for  $i \leftarrow 1$  to  $m$  do
10:    for  $j \leftarrow n$  down to  $1$  do
11:      if  $x_i \neq y_j$  then
12:        continue
13:       $l \leftarrow 1$ 
14:      for  $k \leftarrow \mu_i$  to  $\eta_i$  do
15:        while  $S[k, l][1] < j$  do
16:           $l \leftarrow l + 1$ 
17:           $S[k, l] \leftarrow (i, j)$ 
18:          if  $x_i \geq x_{S[k, l-1][0]}$  then
19:             $Prev[i, j] \leftarrow S[k, l-1]$ 
20:          if  $l > h$  then
21:             $h \leftarrow l$ 
22:            for  $k' \leftarrow 1$  to  $p$  do
23:               $S[k', h+1] \leftarrow (\infty, \infty)$ 
24:            for  $k \leftarrow \eta_i + 1$  to  $p$  do
25:              if  $S[k, l][1] \leq j$  then
26:                break
27:               $S[k, l] \leftarrow (i, j)$ 
28:   return  $h$ 

```

6.2.4. Backtracking

The result of the algorithm is now a triple. It contains the representative point of an LCAIS. We can use the $Prev$ table to reconstruct a partial result. The reason is that while backtracking, we can only get the set of representative points for each of the LCAIS's prefix. However, we can reconstruct the parts of the LCAIS between those representative points. For that we need to solve the *Specialized LCAIS problem*.

If (i_1, j_1) is a representative point of an LCAIS of X_{i_2} and Y_{j_2} , then we can reconstruct the part of LCAIS after the representative point by solving the *Specialized LCAIS Problem* on X_{i_1, i_2} and Y_{j_1, j_2} . Then we can solve the same problem for the representative point before (i_1, j_1) (contained by $Prev[i_1, j_1]$) on X_{i_1-1} and Y_{j_1-1} . The following algorithm describes the whole process. Here, $maxNode$ is the triple returned by the LCAIS algorithm.

```

function BACKTRACK( $maxNode$ )
  ( $\_, top, left$ )  $\leftarrow maxNode$ 
  ( $bottom, right$ )  $\leftarrow (m, n)$ 
   $S \leftarrow \text{SPECIALIZEDLCAIS}(X_{top, bottom}, Y_{left, right})$ 
  while  $top \neq 0$  and  $left \neq 0$  do
    ( $bottom, right$ )  $\leftarrow (top - 1, left - 1)$ 
    ( $top, left$ )  $\leftarrow Prev[top, left]$ 
     $S \leftarrow \text{SPECIALIZEDLCAIS}(X_{top, bottom}, Y_{left, right}) \cdot S$ 
  return  $S$ 

```

6.2.5. Time and space complexity

The backtracking takes $O(mn)$ time because of constructing the *Specialized LCAISs*. It does not change the time complexity of the LCAIS algorithms. The $Prev$ table takes $O(mn)$ space, whereas our algorithms take $O(nc_p)$ space even after making the changes. Hence, our algorithms can reconstruct an LCAIS in $O(mn)$ space.

6.2.6. Changes required for the third approach

Algorithm 6 can be tweaked to achieve the same space complexity ($O(mn)$) without sacrificing anything in time. This tweaked version is shown in Algorithm 7. The main changes are: initializing the $Prev$ matrix (Line 5) and updating both S and $Prev$ matrix (Lines 18–19).

7. Discussions

At this point, a comparative study of our algorithms with the algorithm presented in paper [26] is in order. Note that, none of our algorithms are methodically similar to their algorithm.

However, our first algorithm can be seen as **an easier version to their approach**, as our algorithm is a direct extension of LCS. Both of the approaches keep track of the **length of all LCAISs** of all pairs of prefixes of X and Y . Our method keeps the

length of the LCAISs based on value of the maximum element in it, whereas their method does it based on the index pair that has put the maximum element in the LCAIS.

Now let us compare the results of our algorithm to that of [26]. Here, we suppose $n \leq m$ without loss of generality. Recall that the algorithm in [26] runs in $O(nm\ell)$ time and takes $O(nm)$ space if an LCAIS is to be recovered, otherwise it is $O(n\ell)$.

- When the elements are integers, Algorithm 3 and 4 runs in $O(mn)$ time and $O(n)$ space if only the length of LCAIS is to be calculated, as c is a constant and $c_p \leq c$. This condition does not affect the algorithm of [26] making our algorithms $O(\ell)$ factor better in both time and space. Even if the elements are not integers, our algorithm is better when $c_p < \ell$.
- When the number of matches ($\mathcal{M}$) is linear (e.g., elements are repeated in a sequence at most a constant number of times), Algorithm 6 runs in $O(mn)$ time. This condition does not affect the algorithm of [26] making our algorithms $O(\ell)$ factor faster. Again, when the number of matches ($\mathcal{M}$) is quadratic ($O(mn)$), ℓ has to be linear ($O(n)$), which implies that both $O(mn + \mathcal{M}(p + \ell))$ and $O(nm\ell)$ turn into $O(n^2m)$. Also, both Algorithm 6 and that of [26] take $O(n\ell)$ space if only the length of LCAIS is to be calculated. So, we can say that our third algorithm is never worse.
- Algorithms 2–6 take the same $O(mn)$ space if an LCAIS is to be recovered.

Finally, we can see that all the algorithms in this paper and in [26] have cubic time at worst. It is easy to prove a weak lower bound of quadratic time through reduction from LCAIS to LCIS (by setting $c = 0$). As an immediate future work, we will try to either find a subcubic-time algorithm, or prove that cubic-time is the best anyone can do.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Amir Abboud, Arturs Backurs, Virginia Vassilevska Williams, Tight hardness results for LCS and other sequence similarity measures, in: 2015 IEEE 56th Annual Symposium on Foundations of Computer Science, 2015, pp. 59–78.
- [2] Amir Abboud, Aviad Rubinfeld, Fast and deterministic constant factor approximation algorithms for LCS imply new circuit lower bounds, in: Anna R. Karlin (Ed.), 9th Innovations in Theoretical Computer Science Conference (ITCS 2018), in: Leibniz International Proceedings in Informatics (LIPIcs), vol. 94, Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, ISBN 978-3-95977-060-6, 2018, pp. 35:1–35:14, <http://drops.dagstuhl.de/opus/volltexte/2018/8349>.
- [3] G. de, B. Robinson, On the representations of the symmetric group, Am. J. Math. 60 (3) (1938) 745–760 (ISSN 0002-9327, 1080-6377), <http://www.jstor.org/stable/2371609>.
- [4] Karl Bringman, Marvin Künnemann, Multivariate fine-grained complexity of longest common subsequence, in: Proceedings of the 2018 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2018, pp. 1216–1235, <https://epubs.siam.org/doi/abs/10.1137/1.9781611975031.79>.
- [5] Karl Bringmann, Marvin Künnemann, Quadratic conditional lower bounds for string problems and dynamic time warping, in: 2015 IEEE 56th Annual Symposium on Foundations of Computer Science, 2015, pp. 79–97.
- [6] Yi-Ching Chen, Kun-Mao Chao, On the generalized constrained longest common subsequence problems, J. Comb. Optim. 21 (3) (Aug. 2009) 383–392, <https://doi.org/10.1007/s10878-009-9262-5>.
- [7] Thomas H. Cormen, et al., Introduction to Algorithms, 3rd edition, MIT Press, ISBN 978-0-262-03384-8, 2009, <http://mitpress.mit.edu/books/introduction-algorithms>.
- [8] Lech Duraj, Marvin Künnemann, Adam Polak, Tight conditional lower bounds for longest common increasing subsequence, <https://doi.org/10.1007/s00453-018-0485-7>, July 2018.
- [9] Elmasry Amr, The longest almost-increasing subsequence, in: My T. Thai, Sartaj Sahni (Eds.), Computing and Combinatorics, Springer Berlin Heidelberg, Berlin, Heidelberg, ISBN 978-3-642-14031-0, 2010, pp. 338–347.
- [10] P. van Emde Boas, Preserving order in a forest in less than logarithmic time, in: Proceedings of the 16th Annual Symposium on Foundations of Computer Science, SFCS '75, IEEE Computer Society, USA, 1975, pp. 75–84.
- [11] Peter van Emde Boas, R. Kaas, E. Zijlstra, Design and implementation of an efficient priority queue, Math. Syst. Theory 10 (1977) 99–127, <http://dblp.uni-trier.de/db/journals/mst/mst10.html#BoasKZ77>.
- [12] Michael L. Fredman, On computing the length of longest increasing subsequences, Discrete Math. (ISSN 0012-365X) 11 (1) (1975) 29–35, [https://doi.org/10.1016/0012-365X\(75\)90103-X](https://doi.org/10.1016/0012-365X(75)90103-X), <https://www.sciencedirect.com/science/article/pii/0012365X7590103X>.
- [13] Costas S. Iliopoulos, M. Sohel Rahman, Algorithms for computing variants of the longest common subsequence problem, Theor. Comput. Sci. 395 (2–3) (2008) 255–267, <https://doi.org/10.1016/j.tcs.2008.01.009>.
- [14] Costas S. Iliopoulos, Mohammad Sohel Rahman, A new efficient algorithm for computing the longest common subsequence, Theory Comput. Syst. 45 (2) (2009) 355–371, <https://doi.org/10.1007/s00224-008-9101-6>.
- [15] Costas S. Iliopoulos, Mohammad Sohel Rahman, New efficient algorithms for the LCS and constrained LCS problems, Inf. Process. Lett. 106 (1) (2008) 13–18, <https://doi.org/10.1016/j.ipl.2007.09.008>.
- [16] Costas S. Iliopoulos, et al., Algorithms for computing the longest parameterized common subsequence, in: Bin Ma, Kaizhong Zhang (Eds.), Combinatorial Pattern Matching, 18th Annual Symposium, CPM 2007, London, Canada, July 9–11, 2007, Proceedings, in: Lecture Notes in Computer Science, vol. 4580, Springer, 2007, pp. 265–273.
- [17] Costas S. Iliopoulos, et al., Finite automata based algorithms on subsequences and supersequences of degenerate strings, J. Discret. Algorithms 8 (2) (2010) 117–130, <https://doi.org/10.1016/j.jda.2008.10.004>.
- [18] Costas S. Iliopoulos, et al., The constrained longest common subsequence problem for degenerate strings, in: Jan Holub, Jan Zdárek (Eds.), Implementation and Application of Automata, 12th International Conference, CIAA 2007, Prague, Czech Republic, July 16–18, 2007, Revised Selected Papers, in: Lecture Notes in Computer Science, vol. 4783, Springer, 2007, pp. 309–311.
- [19] Donald E. Knuth, Permutations, matrices, and generalized Young tableaux, Pac. J. Math. 34 (3) (1970) 709–727, <https://doi.org/10.2140/pjm.1970.34.709>.

- [20] Martin Kutz, et al., Faster algorithms for computing longest common increasing subsequences, in: Selected Papers from the 17th Annual Symposium on Combinatorial Pattern Matching (CPM 2006), J. Discret. Algorithms (ISSN 1570-8667) 9 (4) (2011) 314–325, <https://doi.org/10.1016/j.jda.2011.03.013>, <https://www.sciencedirect.com/science/article/pii/S1570866711000438>.
- [21] David Liben-Nowell, Erik Vee, An Zhu, Finding longest increasing and common subsequences in streaming data, in: Lusheng Wang (Ed.), Computing and Combinatorics, Springer Berlin Heidelberg, Berlin, Heidelberg, ISBN 978-3-540-31806-4, 2005, pp. 263–272.
- [22] Edward M. McCreight, Priority search trees, SIAM J. Comput. 14 (2) (1985) 257–276, <https://doi.org/10.1137/0214021>.
- [23] Johra Muhammad Moosa, M. Sohel Rahman, Fatema Tuz Zohora, **Computing a longest common subsequence that is almost increasing on sequences having no repeated elements**, J. Discret. Algorithms (ISSN 1570-8667) 20 (StringMasters 2011 Special Issue) (2013) 12–20, <https://doi.org/10.1016/j.jda.2012.12.005>, <https://www.sciencedirect.com/science/article/pii/S1570866712001633>.
- [24] S. Muthukrishnan, Data streams: algorithms and applications, Found. Trends Theor. Comput. Sci. (ISSN 1551-305X) 1 (2) (Aug. 2005) 117–236, <https://doi.org/10.1561/04000000002>.
- [25] C. Schensted, Longest increasing and decreasing subsequences, Can. J. Math. 13 (1961) 179–191, <https://doi.org/10.4153/CJM-1961-015-3>.
- [26] Toan Thang Ta, Yi-Kung Shieh, Chin Lung Lu, **Computing a longest common almost-increasing subsequence of two sequences**, Theor. Comput. Sci. (ISSN 0304-3975) 854 (2021) 44–51, <https://doi.org/10.1016/j.tcs.2020.11.035>, <https://www.sciencedirect.com/science/article/pii/S0304397520306733>.
- [27] Yin-Te Tsai, The constrained longest common subsequence problem, Inf. Process. Lett. (ISSN 0020-0190) 88 (4) (2003) 173–176, <https://doi.org/10.1016/j.ipl.2003.07.001>, <https://www.sciencedirect.com/science/article/pii/S002001900300406X>.
- [28] I-Hsuan Yang, Chien-Pin Huang, Kun-Mao Chao, A fast algorithm for computing a longest common increasing subsequence, Inf. Process. Lett. (ISSN 0020-0190) 93 (5) (2005) 249–253, <https://doi.org/10.1016/j.ipl.2004.10.014>, <https://www.sciencedirect.com/science/article/pii/S0020019004003242>.